



Securite web – Les bases

Protéger un site sans paniquer.



Mots de passe



HTTPS



Pièges courants



FORMATION SECURITE WEB - LES BASES

Securite web

Protéger un site et ses comptes sans
paniquer, explique simplement.

DanielCraft - Livre debutant - Pack Securite

Sommaire

Clique un chapitre ou un sous-chapitre pour y aller.

1. Chapitre 1 - C'est quoi la securite web ?

- Ce que ce n'est pas
- Ce que tu vas savoir faire
- Comment lire ce livre
- Petite histoire
- Erreur classique
- En vrai
- A toi

2. Chapitre 2 - Menaces courantes

- Phishing et ingenierie sociale
- Vol d'accès et reemploi de secrets
- Oublis : maj et sauvegardes
- Formulaires et entrees non filtrees
- Petite histoire
- Erreur classique
- En vrai
- A toi

3. Chapitre 3 - Mots de passe

- Long et imprevisible
- Unique par service
- Gestionnaire et 2FA
- Petite histoire
- Erreur classique
- En vrai
- A toi

4. Chapitre 4 - HTTPS et le cadenas

- Ce que HTTPS fait
- Ce que HTTPS ne fait pas
- Certificat et hebergeur
- Petite histoire
- Erreur classique
- En vrai
- A toi

5. Chapitre 5 - Phishing : freiner

- Signaux frequents
- Gestes de defense
- Petite histoire
- Erreur classique
- En vrai
- A toi

6. Chapitre 6 - Mises a jour

- Quoi mettre a jour
- Comment ne pas casser
- Petite histoire
- Erreur classique
- En vrai
- A toi

7. Chapitre 7 - Sauvegardes

- Quoi sauvegarder
- Ou et comment
- Tester la restore
- Petite histoire
- Erreur classique
- En vrai
- A toi

8. Chapitre 8 - Permissions et moindres privileges

- Comptes et roles
- Fichiers et services
- Petite histoire
- Erreur classique
- En vrai
- A toi

9. Chapitre 9 - Injections : l'idee (defense).

- Ce qui se passe (sans recette)
- Defense : validation
- Defense : requetes preparees (idee).
- Defense : affichage
- Petite histoire
- Erreur classique
- En vrai
- A toi

10. Chapitre 10 - Formulaires et entrees

- Controles de base
- Spam et abus
- Petite histoire
- Erreur classique
- En vrai
- A toi

11. Chapitre 11 - Sessions et cookies

- Gestes debutant
- Petite histoire
- Erreur classique
- En vrai

- A toi

12. Chapitre 12 - RGPD : l'idée pour débutants

- Minimiser
- Protéger et expliquer
- Petite histoire
- Erreur classique
- En vrai
- A toi

13. Chapitre 13 - Mini-projet : checklist d'un petit site

- Périmètre
- Checklist minimale
- Livrable
- Petite histoire
- Erreur classique
- En vrai
- A toi

14. Chapitre 14 - A retenir

- Ce que ce n'est pas
- Carte rapide
- Petite histoire
- Erreur classique
- En vrai
- A toi

15. Chapitre 15 - Atelier : politique de mots de passe

- Étapes
- Critère de réussite
- Petite histoire
- Erreur classique
- A toi

16. Chapitre 16 - Atelier : lire HTTPS correctement

- Étapes
- Critère de réussite
- Petite histoire
- Erreur classique
- A toi

17. Chapitre 17 - Atelier : checklist avant mise en ligne

- Étapes
- Critère de réussite
- Petite histoire
- Erreur classique
- A toi

18. Chapitre 18 - Erreurs classiques

- [Catalogue utile](#)
- [Petite histoire](#)
- [Erreur classique \(meta\)](#)
- [En vrai](#)
- [A toi](#)

19. [Chapitre 19 - Bonnes pratiques](#)

- [Routine hebdo \(15 min\)](#)
- [Routine mensuelle](#)
- [Ce qu'on ne fait pas ici](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

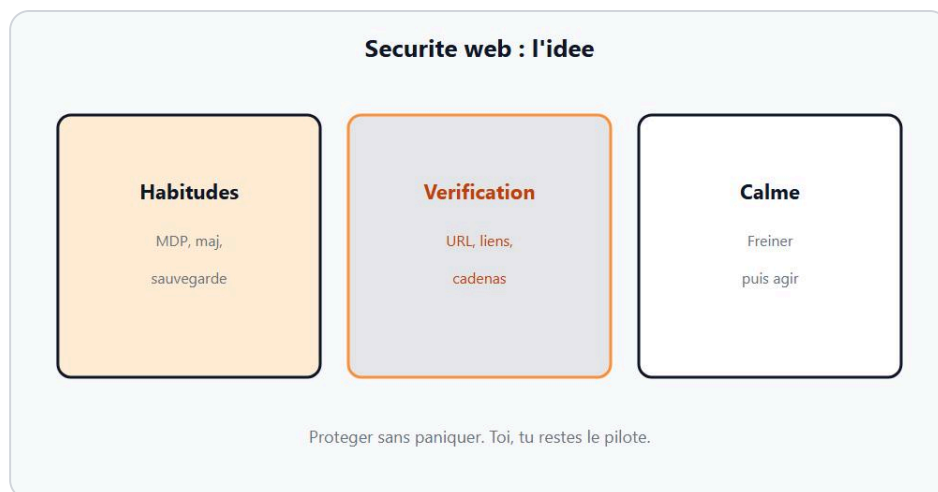
20. [Chapitre 20 - Quiz](#)

- [Questions](#)
- [Corrige court](#)
- [Petite histoire](#)
- [A toi](#)

21. [Chapitre 21 - Bravo](#)

- [Ce que tu peux faire maintenant](#)
- [Ce que tu ne dois pas faire](#)
- [Petite histoire](#)
- [A toi](#)

Chapitre 1 - C'est quoi la securite web ?



Securite web = habitudes simples avant les outils chers.

La **securite web**, pour un debutant, ce n'est pas un badge "hacker ethique" ni une suite d'outils chers. C'est un ensemble d'**habitudes** qui reduisent les accidents : comptes solides, trajet HTTPS, liens regardes deux fois, mises a jour faites, sauvegardes testees, droits limites, entrees de formulaires controlees. Chez DanielCraft, on enseigne la defense comme on enseigne le reste : petit, clair, testable. Lea protege un site artisan. Max a compris le jour ou un meme mot de passe a "saute" sur trois comptes. Sam dit a ses eleves : "Tu n'as pas a tout savoir. Tu as a freiner juste."

Ce livre reste **defense only**. On apprend a reconnaitre, a prevenir, a cocher une checklist. On n'ecrit pas d'exploits, pas de recettes d'attaque, pas de payloads. Si quelqu'un te promet "la faille en cinq minutes", ce n'est pas ce parcours. Ici, tu construis des freins.

★ A RETENIR

Securite web debutant = habitudes repetables qui limitent les degats. Pas de panique. Pas d'attaque a reproduire.

Ce que ce n'est pas

Ce n'est pas de la cryptographie avancee. Ce n'est pas un audit pen-test. Ce n'est pas le RGPD complet (on en donne l'idee). Ce n'est pas "installer vingt plugins et dormir". Un plugin mal tenu peut ouvrir une porte. Ce n'est pas non plus "le cadenas HTTPS = site honnete" : HTTPS protege le trajet, pas ton jugement.

Ce que tu vas savoir faire

A la fin, tu sauras expliquer les menaces courantes sans jargon, choisir une politique de mots de passe, lire un cadenas HTTPS sans magie, freiner face au phishing, planifier mises a jour et sauvegardes, limiter les permissions, comprendre l'idee des injections (validation, requetes preparees) sans jamais exploiter, securiser un formulaire, savoir ce que font sessions et cookies, minimiser les donnees perso, et livrer une checklist pour un petit site. Niveau base. Actionnable demain matin.

Comment lire ce livre

Lis dans l'ordre au debut. Les premiers chapitres posent le sol : idee, menaces, mots de passe, HTTPS, phishing. Puis le quotidien : maj, sauvegardes, permissions. Ensuite le cote "entrees" : injections (idee), formulaires, sessions, RGPD (idee). Le mini-projet assemble. Les ateliers font faire. Le quiz verifie. A chaque fin, un "A toi". Fais-le. Cinq minutes actives battent une lecture passive.



Exemple : Lea verifie le cadenas et l'adresse.

Petite histoire

Lea livrait un site vitrine. Le client a clique un mail "votre domaine expire demain" et a presque colle son mot de passe admin. Lea a freine : "On tape l'URL du registrar soi-meme." Max, lui, utilisait **Artisan2020!** partout. Une fuite ailleurs a donne acces a sa boite mail. Sam a dit : "Unique + gestionnaire. Point." Chez DanielCraft, ces histoires ne servent pas a faire peur. Elles servent a installer un frein.

Erreur classique

Croire que la securite commence par un outil miracle, ou attendre "plus tard" parce qu'on n'est "pas une cible". Les petits sites sont des cibles faciles precisement parce qu'ils sont negliges. Autre piege : tout apprendre d'un coup et ne rien appliquer. Une habitude bat dix intentions.

⚠ ATTENTION

Ce livre ne montre jamais comment attaquer. Si une section te demande d'executer une "preuve d'attaque", ce n'est pas DanielCraft.

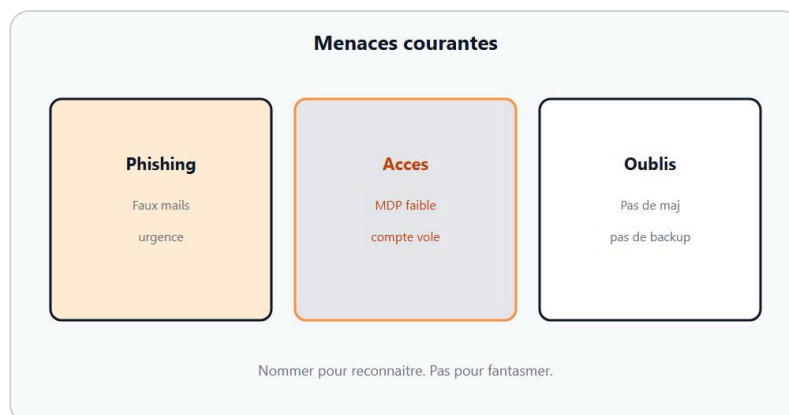
En vrai

Sur une feuille, note trois comptes critiques (mail, hebergeur, admin site) et une chose que tu feras cette semaine pour chacun (mot de passe unique, 2FA, verification URL). Garde ce papier pour le mini-projet.

A toi

Ecris en trois phrases : (1) ce que tu proteges en priorite, (2) ce que tu acceptes d'apprendre d'abord, (3) ce que tu refuses (exploits, recettes d'attaque). Chez DanielCraft, ce petit brief vaut plus qu'une heure de videos floues.

Chapitre 2 - Menaces courantes



Menaces courantes : phishing, vols d'accès, oublis.

Avant les outils, il faut une carte mentale. Les menaces qui touchent le plus souvent un petit site ou un compte debutant ne sont pas des films. Ce sont du **phishing**, des **vols d'accès** (mots de passe reutilises, sessions laisees ouvertes), des **oublis** (pas de mise a jour, pas de sauvegarde), parfois des **formulaire**s mal controles. Lea range ces risques en "humain" et "technique". Max pensait que seul le "hack" comptait. Sam lui a montre qu'un mail urgent et un plugin abandonne font plus de degats que le mythe du genie dans le noir.

Chez DanielCraft, on apprend a **reconnaître** pour **freiner**. On ne detaille pas comment mener une attaque. On nomme le signal, on donne le geste sure.

★ A RETENIR

Menace courante = signal a reconnaître + frein simple. Reconnaître bat paniquer.

Phishing et ingenierie sociale

Quelqu'un se fait passer pour un service (hebergeur, banque, collegue) et pousse a cliquer, a coller un secret, a transferer de l'argent ou des droits. Signaux : urgence, peur, cadeau trop beau, lien qui ne correspond pas au domaine attendu, piece jointe inattendue. Frein : ne pas cliquer, ouvrir le site en tapant l'adresse soi-meme, appeler par un numero connu. Lea lit l'expediteur deux fois. Max survole le lien sans cliquer. Sam refuse les "reponds en 5 minutes ou ton compte saute".

Vol d'accès et reemploi de secrets

Si le meme mot de passe sert au mail et a l'admin WordPress, une fuite ailleurs ouvre deux portes. Les sessions laisees sur un ordi partage font la meme chose sans "piratage". Frein : secrets uniques, gestionnaire, deconnexion, 2FA quand c'est propose. On detaille au chapitre mots de passe.

Oublis : maj et sauvegardes

Une faille publiee reste ouverte tant que tu n'as pas mis a jour CMS, plugins, themes, systeme. Une sauvegarde absente transforme un accident en catastrophe. Frein : calendrier de mises a jour, copie hors machine, test de restauration. Pas glamour. Efficace.

Formulaires et entrees non filtrees

Tout ce que l'utilisateur envoie (champ, fichier, parametre d'URL) peut casser une page ou une requete si on le colle tel quel dans le code ou la base. L'idee - sans jamais montrer d'exploit - est : valider, separer donnees et instructions. Chapitres injections et formulaires.

⚠ ATTENTION

Ne cherche pas "comment faire" une attaque pour "mieux comprendre". Cherche comment la reconnaitre et quoi cocher pour la prevenir.

Petite histoire

Un artisan a reçu un SMS "colis bloqué, payez 1,90 EUR". Max a failli cliquer. Lea a dit : "On ouvre le site du transporteur depuis les favoris." Le vrai suivi montrait rien. Sam a noté le motif : urgence + lien court. Chez DanielCraft, ce motif revient souvent.

Erreur classique

Tout classer en "je suis trop petit pour être visé". Les bots ne lisent pas ton chiffre d'affaires. Autre piège : collectionner des outils antivirus en ignorant mots de passe et mises à jour.

♦ ASTUCE

Fais une liste "mes trois portes" : mail, hébergeur, admin. Si elles tiennent, le reste devient gérable.

En vrai

Pour chaque type (phishing, vol d'accès, oubli maj/backup, formulaire), écris un signal et un frein en une ligne. Tu réutiliseras cette grille au quiz.

A toi

Choisis un mail ou SMS douteux récent (sans cliquer les liens). Décris en cinq lignes pourquoi tu freines. Si tu n'en as pas, invente un scénario "compte expire demain" et écris le frein.

Chapitre 3 - Mots de passe



Mot de passe long, unique, dans un gestionnaire.

Un mot de passe n'est pas un slogan cute. C'est une **cle**. Si la cle est courte, previsible, ou reutilisee partout, une seule fuite ouvre plusieurs portes. Chez DanielCraft, la regle debutant est simple a retenir et dure a tricher : **long**, **unique**, range dans un **gestionnaire**, avec **2FA** quand le service le propose. Lea a migre ses comptes critiques en un week-end. Max a arrete **PrenomAnnee!**. Sam refuse les post-it sous le clavier et les fichiers `mdp.txt` sur le bureau.

On ne te donne pas de listes de mots de passe a casser. On te donne une politique.

★ A RETENIR

Long + unique + gestionnaire (+ 2FA si possible). Un secret reutilise = plusieurs portes ouvertes.

Long et imprevisible

La longueur aide. Une phrase de passe longue, ou un secret genere par le gestionnaire, bat un mot court "complexe" du type `A1b!`. Tu n'as pas a le memoriser pour chaque site : le gestionnaire le retient. Toi, tu memorises le maitre (tres solide) et tu actives le verrouillage de l'appareil.

Unique par service

Mail,hebergeur, admin site, banque, reseaux : chacun son secret. Si un forum fuit, ton admin ne tombe pas avec. Lea tient un inventaire des comptes critiques. Max commence par les trois portes. Sam dit : "Pas de 'je change tout demain'. Trois aujourd'hui."

Gestionnaire et 2FA

Un gestionnaire (application dediee) genere, stocke, remplit. La double authentification ajoute un second facteur (application, cle, parfois SMS avec limites). Ce n'est pas magique. Ca coupe beaucoup de vols d'accès apres fuite de mot de passe. Active-la d'abord sur le mail : c'est souvent la cle de reset de tout le reste.

⚠ ATTENTION

Ne stocke pas tes secrets dans un mail a toi-meme, un Drive partage, un Slack, un README du projet. Le depot Git n'est pas un coffre.



Exemple : Max refuse de réutiliser le même secret.

Petite histoire

Max a réutilisé le même secret "solide" sur boutique et mail. Une fuite boutique a permis une tentative de reset mail. Lea a freiné avec la 2FA déjà active. Sam a fait migrer Max vers un gestionnaire le soir même. Chez DanielCraft, on célèbre le frein, pas la peur.

Erreur classique

Changer tous les mots de passe tous les 30 jours vers des variantes prévisibles (Hiver2024, Hiver2025). Préférer unique + long + 2FA. Autre piège : partager un compte "équipe" avec un seul login admin.

♦ ASTUCE

Commence par mail + hébergeur + admin. Ensuite le reste au fil de l'eau quand tu te connectes.

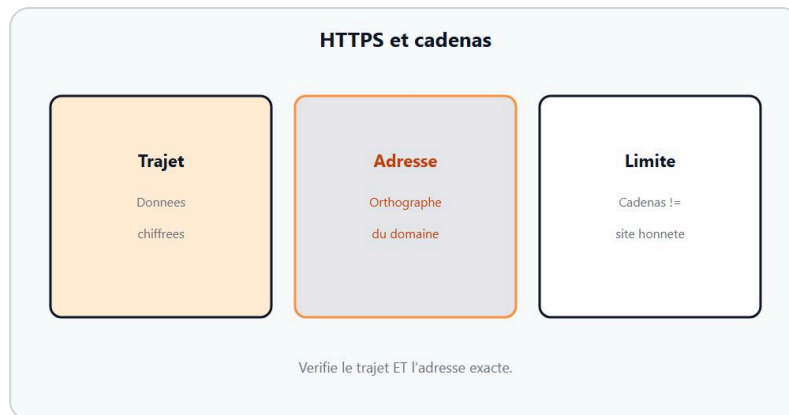
En vrai

Ouvre (ou installe) un gestionnaire. Crée une entrée pour un compte non critique en test. Note la règle que tu appliqueras aux trois comptes critiques.

A toi

Écris ta politique en quatre puces : longueur, unicité, stockage, 2FA. Coche ce qui est déjà vrai. Planifie les trous cette semaine.

Chapitre 4 - HTTPS et le cadenas



HTTPS protege le trajet. Ce n'est pas toute la securite.

HTTPS protege le **trajet** entre ton navigateur et le serveur : les donnees sont chiffrees en transit, plus difficiles a lire ou modifier sur le chemin. Le cadenas (ou l'indicateur equivalent) signale en general que cette protection de trajet est active. Chez DanielCraft, on enseigne HTTPS comme une brique necessaire - pas comme un certificat de saintete. Lea verifie le domaine autant que le cadenas. Max a cru qu'un cadenas rendait un site "sur a 100 %". Sam a corrige : "Sur le trajet, oui. Sur l'intention du site, non."

★ A RETENIR

HTTPS = trajet protege. Utile et attendu. Ce n'est pas toute la securite, ni une preuve d'honnetete.

Ce que HTTPS fait

Il reduit le risque qu'un intermediaire lise ton mot de passe ou ta session sur un Wi-Fi douteux. Il aide aussi a eviter certaines modifications du contenu en transit. Pour un site qui demande login, paiement, ou donnees perso, HTTPS n'est plus optionnel : c'est le minimum.

Ce que HTTPS ne fait pas

Il ne garantit pas que le site est le bon service (regarde le **domaine**). Il ne protege pas un serveur mal configure, un plugin abandonne, un mot de passe faible. Un faux site peut aussi avoir HTTPS. Lea tape l'URL connue. Max compare `boutique-exemple.fr` et une variante avec un caractere en trop. Sam refuse les raccourcis suspects meme avec cadenas.

Certificat et hebergeur

Chez la plupart des hebergeurs modernes, activer HTTPS (Let's Encrypt ou equivalent) est un clic ou une option. Pour un petit site, l'objectif est simple : forcer HTTPS, rediriger HTTP vers HTTPS, verifier que le cadenas apparait sur les pages sensibles. Pas besoin de devenir expert certificats X.509 pour commencer.

◆ ASTUCE

Bookmark les sites critiques (banque, hebergeur, admin). Tu ouvres le favori, tu ne colles pas un lien recu.

Petite histoire

Lea a livré un site encore en HTTP "parce que c'est juste une vitrine". Le formulaire contact envoyait le message en clair sur le trajet. Elle a activé HTTPS avant la mise en prod. Max a vu le cadenas et a souri. Sam a ajouté : "Maintenant, vérifie aussi les mots de passe admin." Chez DanielCraft, une brique après l'autre.

Erreur classique

Confondre "cadenas vert / présent" et "je peux faire confiance aveuglement". Autre piège : ignorer l'avertissement du navigateur "connexion non privée" pour "avancer vite".

⚠ ATTENTION

Si le navigateur crie, tu t'arrêtes. Tu ne "continues quand même" que si tu comprends vraiment pourquoi - et rarement sur un compte critique.

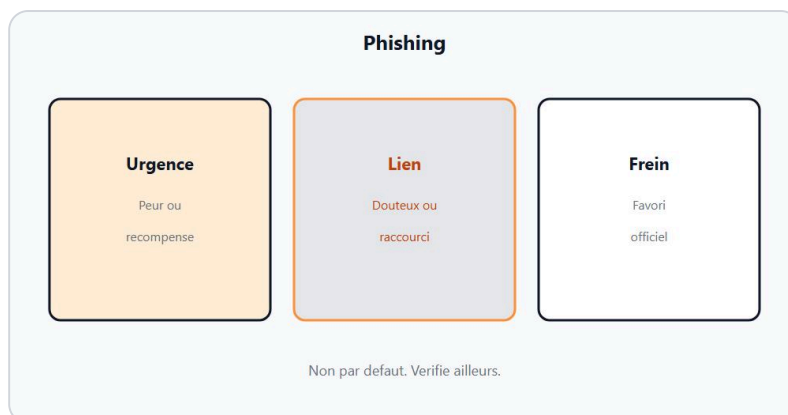
En vrai

Ouvre ton site (ou un site que tu gères). Note : HTTPS oui/non, redirection HTTP, domaine exact. Si non, planifie l'activation avec l'hébergeur.

A toi

Explique à un ami, en trois phrases, ce que HTTPS protège et ce qu'il ne protège pas. Si tu peines, relis ce chapitre avant l'atelier HTTPS.

Chapitre 5 - Phishing : freiner



Phishing : urgence + lien douteux = frein.

Le **phishing** gagne sur la vitesse. Un message urgent, un lien, une peur (compte ferme, facture, colis, domaine). Toi, tu gagnes en **ralentissant**. Chez DanielCraft, le geste sure n'est pas "devenir expert en headers d'email". C'est : ne pas cliquer, verifier a la source, ne jamais coller un secret depuis un message. Lea lit deux fois. Max survole sans ouvrir. Sam enseigne le rituel a voix haute.

★ A RETENIR

Urgence + lien + demande de secret = frein. Tape l'URL toi-meme. Verifie hors message.

Signaux frequents

Urgence ("dans 1 heure"), peur ou cupidite, expéditeur presque bon, lien qui ne matche pas le domaine attendu, piece jointe inattendue, faute volontaire pour presser, demande de mot de passe / code / seed / IBAN. Un site phishing peut avoir un bel HTTPS. Le cadenas ne te sauve pas si le domaine est faux.

Gestes de defense

1. 1. Ne clique pas le lien du message.
2. 2. Ouvre le service via favori ou URL tapee.
3. 3. Si besoin, contacte le support avec un numero / canal deja connu.
4. 4. Signale / supprime le message.
5. 5. Si tu as deja clique ou colle un secret : change le mot de passe (depuis le vrai site), active 2FA, verifie les sessions.

On ne te montre pas comment fabriquer un faux mail. On te montre comment le laisser mourir.

⚠ ATTENTION

Personne de legitime ne devrait te demander ton mot de passe par mail ou SMS. Les codes 2FA non plus, a coller "pour verification".



Exemple : Sam ignore le faux message de compte.

Petite histoire

Sam a reçu un "votre abonnement hebergeur expire, mettez a jour la carte". Le lien menait a une page presque identique. Il a ferme, ouvert le panneau via favori : rien a payer. Lea a note le domaine piege pour la formation. Max a admis qu'il aurait clique "parce que ca faisait vrai". Chez DanielCraft, on normalise le frein, pas la honte.

Erreur classique

"Je survole juste pour voir." Sur mobile, un doigt suffit. Autre piege : repondre pour "verifier si c'est vrai" - tu confirmes que l'adresse est active.

♦ ASTUCE

Cree une phrase personnelle : "Si c'est urgent, je ralentis." Dis-la avant chaque clic douteux.

En vrai

Sans cliquer, observe la boite spam ou les SMS. Liste trois motifs qui reviennent. Ajoute le frein correspondant a ta checklist.

A toi

Redige un mini-protocole phishing en cinq lignes pour toi ou ton equipe. Garde-le a cote de l'ecran.

Chapitre 6 - Mises a jour



Mises a jour : fermer les portes deja connues.

Une mise a jour ennuyeuse ferme souvent une **porte deja connue**. CMS, plugins, themes, extensions navigateur, systeme, runtime : les correctifs sortent parce qu'un risque a ete documente. Tant que tu n'appliques pas, la porte reste ouverte. Chez DanielCraft, on ne romantise pas la maj. On la planifie. Lea met a jour le week-end avec sauvegarde avant. Max a appris apres un plugin abandonne. Sam dit : "Petit, regulier, teste."

★ A RETENIR

Maj = fermer des portes publiees. Calendrier + sauvegarde avant > improvisation le jour de panique.

Quoi mettre a jour

- Systeme et navigateur
- CMS / framework et dependances
- Plugins / themes / extensions
- Bibliotheques du projet (quand tu codes)

Pour un site "cle en main", commence par le panneauhebergeur et le tableau de bord CMS. Desactive ou retire ce qui n'est plus maintenu.

Comment ne pas casser

Sauvegarde avant. Lis les notes courtes si disponibles. Mets a jour d'abord en preprod / staging si tu en as une. Sinon, maj hors heures de pointe et verifie les pages cles (accueil, contact, login, paiement). Lea a une checklist de cinq URL. Max chronometre quinze minutes de verif.

♦ ASTUCE

Note la date de derniere maj sur une ligne dans ton carnet projet. Visible = moins oublie.

Petite histoire

Un theme "gratuit magnifique" n'avait plus de mises a jour depuis deux ans. Lea l'a remplace par un theme simple maintenu. Le site etait moins "wow", plus respirable. Sam a valide. Max a compris que joli sans entretien est un risque. Chez DanielCraft, durable bat tape-a-l'oeil.

Erreur classique

Ignorer les maj "parce que ca marche encore". Autre piege : tout mettre a jour sans sauvegarde, puis paniquer.

ATTENTION

Un composant abandonne n'est pas "stable". C'est souvent "plus corrige".

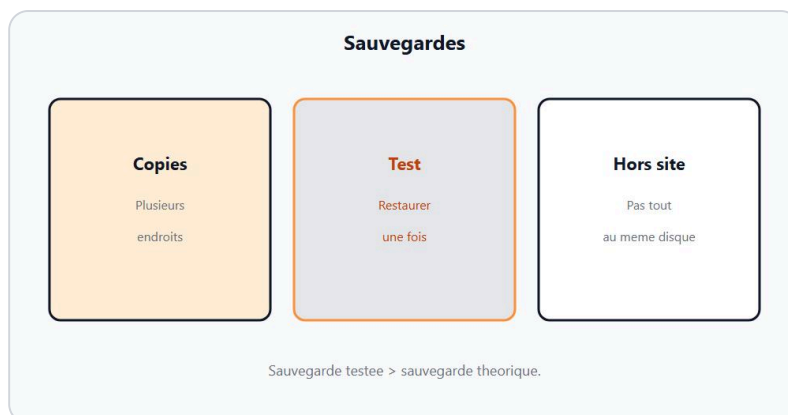
En vrai

Liste tes composants (CMS, 3 plugins max a garder, theme). Coche : a jour / a maj / a retirer.

A toi

Planifie la prochaine fenetre de maj (jour + heure + backup avant). Ecris-la. Fais-la.

Chapitre 7 - Sauvegardes



Sauvegarde = plan B avant le jour J.

Une sauvegarde est un **plan B** decide avant le jour J. Panne disque, fausse manip, ransomware, hebergeur qui plante, maj qui casse : sans copie, tu negocies avec le vide. Chez DanielCraft, une sauvegarde qui n'a jamais ete **restauree en test** est un espoir, pas une assurance. Lea teste une restore tous les mois. Max a cru que "l'hebergeur sauvegarde" suffisait - jusqu'au jour ou la retention etait trop courte. Sam exige : copie hors machine + test.

★ A RETENIR

Sauvegarde = copie + hors site + test de restauration. Sinon, illusion.

Quoi sauvegarder

Fichiers du site (code, media) et base de donnees si tu en as une. Configs critiques (sans coller les secrets dans un repo public). Pour un site simple : export CMS + telechargement fichiers, ou outil hebergeur documente.

Ou et comment

Au moins une copie **ailleurs** que le serveur live (autre stockage, autre compte, disque offline selon le cas). Rotation : plusieurs points dans le temps si possible (hier, semaine derniere). Chiffre / protege l'accès a ces copies : une sauvegarde lisible par tout le monde est une fuite en attente.

⚠ ATTENTION

Ne publie jamais une archive de sauvegarde sur un lien public "temporaire".

Tester la restore

Une fois par mois (ou apres un gros changement) : restaure sur un environnement de test ou verifie que l'archive s'ouvre et que la base importe. Lea chronometre. Max note "OK restore" avec la date. Sam refuse "on verra le jour ou ca casse".

Petite histoire

Max a écrasé une page importante. La sauvegarde de la veille l'a sauvé en vingt minutes. Lea a sourit : "C'est pour ça qu'on teste." Chez DanielCraft, la sauvegarde n'est pas une slide. C'est un réflexe.

Erreur classique

Une seule copie sur le même disque que le site. Autre piège : sauvegarder mais jamais vérifier.

♦ ASTUCE

Ajoute "test restore" dans le même calendrier que les mises à jour.

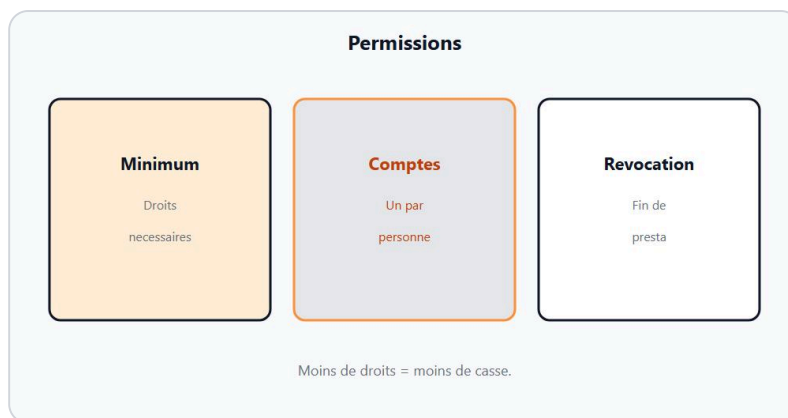
En vrai

Vérifie où sont tes copies aujourd'hui. Si tu n'en as pas, crée la première (même imparfaite) puis améliore.

A toi

Écris ta règle 3-2-1 adaptée débutant : au moins une copie récente hors serveur live, et une date de prochain test restore.

Chapitre 8 - Permissions et moindres privileges



Moins de droits = moins de casse si un compte fuit.

Donner a tout le monde le compte **admin** "parce que c'est plus simple" est un accélérateur de catastrophe. Si ce compte fuit, tout fuit. Le principe du **moindre privilege** : chaque personne ou service a seulement les droits nécessaires a sa tâche. Chez DanielCraft, c'est une habitude, pas un slogan. Lea a un compte éditeur pour le quotidien et un admin rare. Max a arrêté de partager le même login. Sam crée des rôles.

★ A RETENIR

Moins de droits = moins de casse si un compte ou une session fuit.

Comptes et rôles

Admin / propriétaire : installations, utilisateurs, réglages critiques. Éditeur / auteur : contenu. Lecteur : consultation. Pour un freelance et un client : séparez les accès. Désactivez les comptes partis. Révoquez les clés API inutiles.

Fichiers et services

Sur un hébergeur, évitez les dossiers world-writable "parce que ça marche". Limite qui peut déployer. Les clés SSH et tokens CI sont des permissions : rangez-les, rotatez-les si fuite. On reste débutant : pas besoin d'IAM cloud avancé pour commencer - juste ne pas tout ouvrir.

♦ ASTUCE

Une fois par trimestre : liste des comptes avec accès admin. Qui est encore légitime ?

Petite histoire

Un stagiaire avait l'admin "le temps du stage". Trois mois plus tard, le compte existait encore. Lea l'a révoqué le jour de l'audit maison. Sam a ajouté une case "offboarding" à la checklist. Max a compris que permission = dette si on oublie.

Erreur classique

Un seul compte pour toute l'équipe. Autre piège : laisser des comptes de test avec mots de passe faibles sur le live.

ATTENTION

Les comptes "demo123" / "test/test" sur la prod sont des portes ouvertes avec neon.

En vrai

Dresse la liste des gens (et bots) qui ont acces a ton site. Pour chacun : role minimum. Raye le superflu.

A toi

Ecris qui a l'admin aujourd'hui et pourquoi. Si la reponse est "tout le monde", corrige avant la fin de semaine.

Chapitre 9 - Injections : l'idée (defense)



Entrees non filtrees : risque. Validation + requetes preparees.

Quand une application **colle** une entree utilisateur (champ, URL, fichier) directement dans une instruction - requete base de donnees, page HTML, commande - cette entree peut **casser** le sens de l'instruction. L'idée s'appelle souvent **injection**. Chez DanielCraft, on explique le risque pour **prevenir**. On ne montre **aucun** exploit, **aucune** payload, **aucune** recette d'attaque. Lea valide les champs. Max a appris a ne plus concatener une requete a la main. Sam enseigne : "Separe les donnees et les instructions."

★ A RETENIR

Entree non fiable = ne jamais la coller telle quelle dans une requete ou une page. Valider. Parametrer. Echapper a l'affichage.

Ce qui se passe (sans recette)

Imagine une question a une base : tu veux chercher un nom. Si tu construis la question en collant le texte tape par l'utilisateur au milieu, ce texte peut etre interprete comme une partie de la question plutot que comme une simple donnee. Resultat possible : comportement inattendu, fuite, corruption. Pareil cote page : un texte affiche sans precaution peut devenir du code de page. Tu n'as pas besoin de savoir "comment faire" pour comprendre "il ne faut pas coller".

Defense : validation

Verifie le type, la longueur, le format attendu (email, nombre, liste de valeurs). Refuse ce qui sort du contrat. La validation cote navigateur aide l'UX. La validation **serveur** est le vrai frein.

Defense : requetes preparees (idee)

Les **requetes parametrees** / preparees envoient la structure de la question d'un cote et les valeurs de l'autre. La base traite la valeur comme donnee, pas comme code SQL. C'est l'idée a retenir. Utilise les outils de ton langage / ORM qui le font correctement. N'invente pas une concatenation "securisee a la main" si tu debutes.

Defense : affichage

Quand tu reaffiches une entree sur une page, utilise les mecanismes d'echappement du framework (ou equivalent) pour que le texte reste du texte. Encore une fois : pas de payload de demo ici.

⚠ ATTENTION

Ce chapitre est strictement defensif. Pas de PoC. Pas de "pour tester chez toi". Si tu veux approfondir, cherche des ressources defense / OWASP dans un cadre legitime - pas des tutoriels d'attaque.

Petite histoire

Max concatenait une recherche "pour aller vite". Lea a remplace par une requete parametree du framework. Sam a fait valider longueur et alphabet autorise. Aucun d'eux n'a "teste une injection". Ils ont ferme la porte. Chez DanielCraft, c'est le succes.

Erreur classique

Croire que "personne ne visitera mon petit form". Autre piege : montrer des payloads "pedagogiques" a une equipe junior - ca devient une arme mal rangee.

♦ ASTUCE

Checklist code : entree -> valider -> API parametree / echappement -> stocker / afficher.

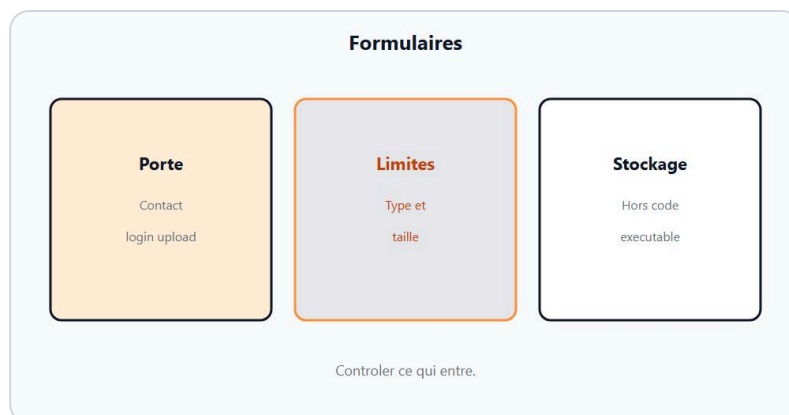
En vrai

Repere un endroit de ton projet (ou un schema) ou une entree arrive. Ecris comment tu valides et comment tu evites de coller dans une instruction.

A toi

Explique en cinq phrases a un ami : pourquoi coller une entree dans une requete est dangereux, et quels freins tu utilises - sans donner d'exemple d'attaque.

Chapitre 10 - Formulaires et entrees



Formulaire = porte d'entree. Controler ce qui arrive.

Un **formulaire** est une porte. Contact, inscription, upload, recherche : tout ce qui entre doit etre **attendu**, **limite**, **controle cote serveur**. Chez DanielCraft, on ne fait pas confiance au seul navigateur. Lea valide email et longueur. Max a laisse un champ fichier sans type - mauvaise idee. Sam ajoute un token anti-CSRF quand le framework le propose, et un rate limit simple sur les spams.

★ A RETENIR

Formulaire = porte. Controle serveur obligatoire. Client = aide, pas autorite.

Controles de base

- Champs requis vs optionnels clairs
- Longueur max
- Format (email, telephone, choix liste)
- Types de fichiers autorises et taille max si upload
- Messages d'erreur sans reveler trop d'interne

Stocke le minimum. N'enregistre pas un numero de carte dans un champ contact "au cas ou".

Spam et abus

Captcha raisonnable, limitation de debit, moderation : selon le contexte. Un formulaire contact ouvert sans frein devient une boite a spam. Lea a ajoute un delai simple et une validation email. Max filtre cote serveur les messages vides / liens en masse.

♦ ASTUCE

Liste les champs vraiment utiles. Chaque champ en moins est une surface en moins.



Exemple : Max valide avant d'enregistrer.

Petite histoire

Le formulaire "devis" demandait pièce d'identité "pour aller plus vite". Lea a retiré : hors finalité. Sam a rappelé le chapitre RGPD. Max a gardé nom, email, message. Le site respirait. Chez DanielCraft, minimiser est aussi de la sécurité.

Erreur classique

Valider seulement en JavaScript. Autre piège : accepter n'importe quel upload "image" sans vérifier type/taille.

⚠ ATTENTION

Un fichier uploadé mal contrôlé peut devenir une porte. Reste strict sur types et emplacement de stockage.

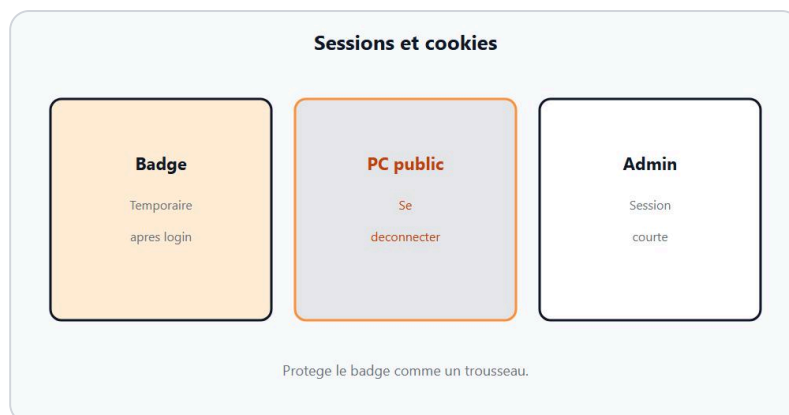
En vrai

Prends un formulaire de ton site. Pour chaque champ : type attendu, longueur, obligatoire, ou ça part (mail, base). Complète les trous.

A toi

Ecris la checklist formulaire (8 cases max) que tu recocheras à chaque nouveau form.

Chapitre 11 - Sessions et cookies



Sessions et cookies : qui est connecte, jusqu'a quand.

Une **session** dit "cet utilisateur est deja authentifie pour un temps". Un **cookie** est souvent le jeton que le navigateur renvoie pour retrouver cette session. Chez DanielCraft, on veut des sessions **courtes autant que possible**, transportees en **HTTPS**, avec **deconnexion** claire, sans laisser un poste partage "encore connecte". Lea se deconnecte sur les ordi d'atelier. Max a compris apres avoir laisse l'admin ouvert en bibliotheque. Sam explique HttpOnly / Secure comme idees : le framework serieux les active souvent pour toi.

★ A RETENIR

Session = preuve temporaire. Cookie = jeton. HTTPS, duree, deconnexion : freins de base.

Gestes debutant

- Preferer HTTPS partout ou il y a login
- Se deconnecter sur machine partagee
- Ne pas cocher "rester connecte" sur un PC public
- Invalider les sessions apres changement de mot de passe (si l'outil le permet)
- Limiter les comptes admin ouverts en parallele

Tu n'as pas a reecrire un moteur de session. Tu as a utiliser proprement celui du CMS / framework.

⚠ ATTENTION

Un cookie de session n'est pas un souvenir a partager. Ne l'envoie pas par mail "pour debug" sur un canal large.

Petite histoire

Max a demontre une feature admin sur le laptop d'un cafe, puis est parti sans logout. Lea a change le mot de passe et force la deconnexion depuis le panneau. Sam a ajoute "logout" a la fin de chaque atelier. Chez DanielCraft, le detail sauve.

Erreur classique

Croire que fermer l'onglet = deconnexion complete partout. Autre piege : sessions admin qui ne meurent jamais.

♦ ASTUCE

Après un voyage / PC prête : change le mdp admin et vérifie les sessions actives si l'outil le montre.

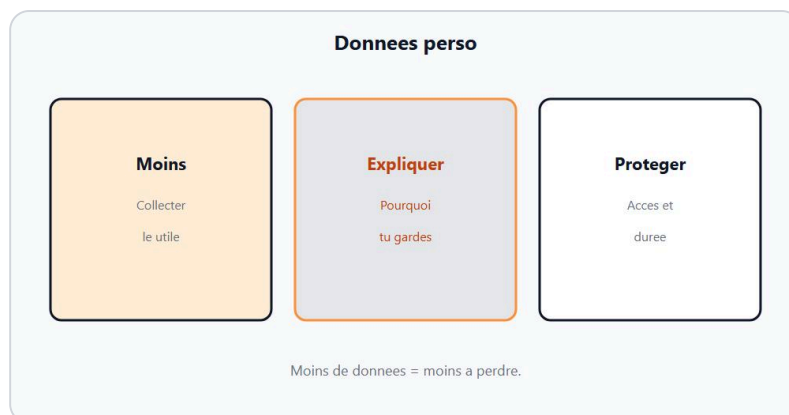
En vrai

Sur ton CMS ou app, trouve où se reglent durée de session / deconnexion / "se souvenir de moi". Note le réglage actuel.

A toi

Ecris trois règles perso (ex. : pas d'admin en Wi-Fi café sans VPN/prudence, logout atelier, 2FA mail). Affiche-les.

Chapitre 12 - RGPD : l'idée pour debutants



Donnees perso : minimiser, proteger, expliquer.

Le **RGPD** encadre le traitement des **donnees personnelles** en Europe. Ce chapitre donne une **boussole**, pas un conseil juridique personnalise. Idées a retenir : **finalite** claire, **minimisation**, **securite**, information des personnes, droits d'accès / suppression selon le cas, responsabilite. Chez DanielCraft, on relie ça a la technique : moins tu stockes, moins tu fuis. Lea retire les champs inutiles. Max arrete de garder des CSV "au cas ou" sur le bureau. Sam rappelle : l'IA et les formulaires n'effacent pas le cadre.

★ A RETENIR

Donnees perso : pourquoi tu les as, le minimum, protegees, expliquees. Ceci n'est pas un audit legal.

Minimiser

Tu n'as pas besoin de la date de naissance pour un devis simple. Tu n'as pas besoin du numero de securite sociale pour une newsletter. Remplace par ce qui sert vraiment. Moins de colonnes = moins de surface.

Protéger et expliquer

Accès limites, HTTPS, mots de passe, sauvegardes protegees. Une page / mention d'information adaptee a ton cas (fais-toi aider si tu es pro). Durees de conservation : ne garde pas "pour toujours" sans raison.

⚠ ATTENTION

Coller un export clients dans un chat public ou un prompt cloud sans cadre = risque. Anonymise / minimise avant.

Petite histoire

Un client voulait "tout savoir sur chaque visiteur". Lea a propose : stats agregees + formulaire contact volontaire. Sam a valide l'idée minimisation. Max a archive puis efface un vieux tableur inutile. Chez DanielCraft, sobriete = hygiène.

Erreur classique

Croire "c'est mon site donc je fais ce que je veux des emails". Autre piège : confondre anonymisation vraie et pseudonyme facile a re-identifier.

♦ ASTUCE

Pour chaque champ stocke, écris la finalité en une phrase. Si tu bloques, retire le champ.

En vrai

Liste les données perso que ton projet touche. Pour chacune : finalité, ou c'est stocké, qui y accède.

A toi

Rédige un paragraphe "ce que nous collectons et pourquoi" en langage clair (brouillon). Fais-le relire par quelqu'un de compétent si tu publies en vrai.

Chapitre 13 - Mini-projet : checklist d'un petit site



Checklist securite d'un petit site.

Objectif : produire une **checklist securite** concrete pour un site vitrine ou petit projet, puis la **cocher** avec preuves courtes (oui/non + note). Pas d'attaque. Pas de scan agressif non autorise. Chez DanielCraft, livrer = document utilisable. Lea anime. Max coche. Sam challenge les "on verra".

★ A RETENIR

Une checklist cochee avec dates bat dix bonnes intentions.

Perimetre

Choisis un site que tu as le droit d'administrer (ou un projet perso). Note l'URL, l'hebergeur, le CMS eventuel. Fixe une date de revue.

Checklist minimale

1. 1. HTTPS actif + redirection HTTP
2. 2. Comptes : MDP uniques gestionnaire + 2FA mail/admin si possible
3. 3. Moindre privilege : qui a l'admin ?
4. 4. Mises a jour CMS/plugins/theme a jour ou planifiees
5. 5. Sauvegarde recente + date du dernier test restore
6. 6. Formulaires : validation serveur, champs minimaux
7. 7. Pas de comptes test sur le live
8. 8. Rituel phishing : favoris / URL tapee pour les services critiques
9. 9. Mentions / minimisation donnees (idee RGPD)
10. 10. Page "incident" : qui contacter, changer quels secrets

Livrable

Une page (Notion, markdown, papier photo) avec les 10 lignes, statut, date, responsable. Bonus : capture (floutee) du cadenas / panneau maj.

Petite histoire

Lea, Max et Sam ont fait la checklist sur un site asso en quarante minutes. Trois trous : plugin mort, backup jamais testee, compte stagiaire. Ils ont corrige le soir. Chez DanielCraft, c'est un mini-projet reussi.

Erreur classique

Transformer le mini-projet en "audit pirate". Autre piege : cocher tout vert sans verifier.

ATTENTION

Reste sur ton perimetre. Pas de tests d'intrusion improvises sur le site d'autrui.

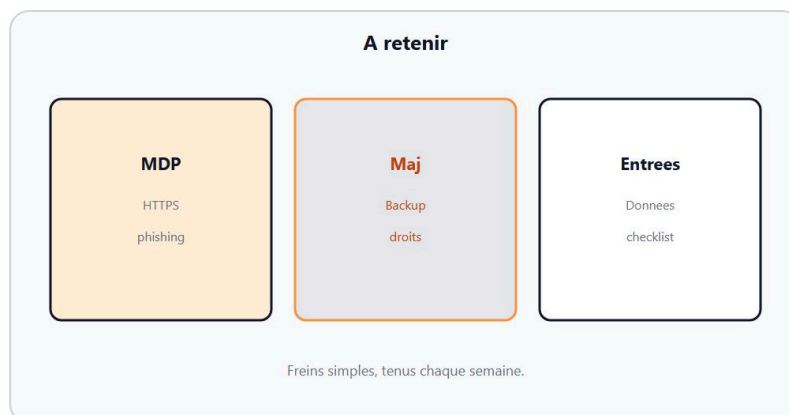
En vrai

Remplis la checklist maintenant pour un vrai site. Les cases rouges deviennent ton plan de semaine.

A toi

Livre le document. Partage-le a la personne responsable du site. Planifie la prochaine revue (90 jours).

Chapitre 14 - A retenir



MDP, HTTPS, maj, sauvegarde, mefiance liens.

Tu as traverse le socle securite web debutant. Ce chapitre range. Chez DanielCraft, on fait une carte avant les ateliers. Lea coches ce qu'elle reutilise chaque semaine. Max relit cette page avant une mise en ligne. Sam s'en sert comme checklist orale.

Securite web base = **habitudes** : mots de passe longs et uniques dans un gestionnaire (+ 2FA), **HTTPS** compris sans magie, **frein phishing**, **mises a jour** planifiees, **sauvegardes** testees, **permissions** minimales, **entrees** validees et requetes preparees (idee), formulaires controles, sessions propres, donnees perso minimisees. Defense only. Pas d'exploits.

★ A RETENIR

MDP, HTTPS, maj, backup, freiner les liens : la carte. Le reste est de la pratique.

Ce que ce n'est pas

Ce n'est pas la fin de la securite. Audits, WAF, threat modeling avance : hors scope. Ce n'est pas un examen legal RGPD. Si une case est floue, retourne au chapitre. Lea dit : "trois freins solides battent vingt outils cites".

Carte rapide

- Habitudes > outils miracles
- Menaces : reconnaitre + freiner
- MDP long, unique, gestionnaire, 2FA
- HTTPS = trajet, pas saintete
- Phishing : ralentir, URL tapee
- Maj + composants maintenus
- Backup hors site + restore test
- Moindre privilege
- Injections : valider / parametrier / echapper (sans exploit)
- Formulaires : serveur autorite
- Sessions : logout, HTTPS, duree

- RGPD idee : minimiser
- Checklist site livree

♦ ASTUCE

Pour chaque item, cite un geste de ton site. Si tu ne peux pas, relis le chapitre.

Petite histoire

Après le mini-projet, Max a dit "en fait c'est surtout de la discipline". Lea a sourit. Sam a demandé cinq mots. Les élèves : freiner, unique, HTTPS, maj, backup. Pas "hacker". Chez DanielCraft, ces cinq mots suffisent pour repartir.

Erreur classique

Tout mémoriser avant d'agir. Ou tout sauter pour "acheter un antivirus miracle". La carte sert à prioriser.

⚠ ATTENTION

Si tu ne retiens qu'une chose : ralentis sur les liens, solidifie les trois portes (mail, hébergeur, admin).

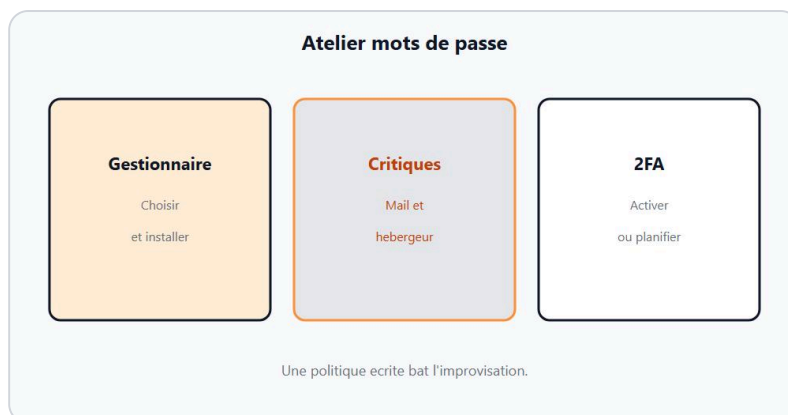
En vrai

Sans notes, écris les dix idées que tu gardes. Compare à la liste. Entoure les trous.

A toi

Imprime ou copie la carte. Coches vert / orange / rouge. Traite un orange aujourd'hui.

Chapitre 15 - Atelier : politique de mots de passe



Atelier : politique de mots de passe.

Duree visee : 30 a 45 minutes. Objectif : ecrire ta politique et l'appliquer a **trois comptes critiques**. Pas de partage de secrets dans le rendu. Chez DanielCraft, on livre une regle et des cases cochees, pas les mots de passe.

★ A RETENIR

Inventaire + uniques + gestionnaire + 2FA sur le mail d'abord.

Etapes

1. Liste tes comptes critiques (mail, hébergeur, admin site, banque...).
2. Installe / ouvre un gestionnaire.
3. Rédige la politique (longueur, unicite, stockage, 2FA).
4. Migre trois comptes : nouveau secret unique, 2FA si dispo.
5. Raye les post-it / fichiers `mdp.txt` si tu en avais.

Critere de reussite

Politique écrite + trois migrations faites + mail en 2FA (ou plan datee si le service bloque).

⚠ ATTENTION

Ne colle aucun secret dans un chat, un ticket, un commit.

Petite histoire

Max a migre mail, OVH-like, et admin en une soiree. Lea a verifie la 2FA mail. Sam a refuse le screenshot du gestionnaire ouvert en atelier. Chez DanielCraft, discretion = competence.

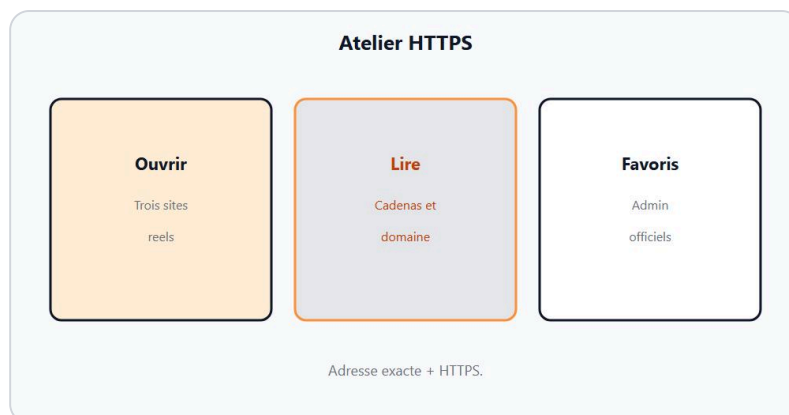
Erreur classique

Changer vers des variantes (`Ete2026!`). Autre piege : tout reporter a "ce week-end" sans date.

A toi

Coche : politique OK, 3 comptes OK, 2FA mail OK. Si non, bloque l'atelier suivant jusqu'a correction.

Chapitre 16 - Atelier : lire HTTPS correctement



Atelier : lire un cadenas correctement.

Duree : 20 a 30 minutes. Objectif : observer sans magie. Ouvre trois sites (dont le tien si possible). Pour chacun : domaine exact, HTTPS oui/non, avertissement navigateur, pages login en HTTPS. Chez DanielCraft, on entraine l'oeil, pas le clic rapide.

★ A RETENIR

Cadenas utile. Domaine verifie. HTTPS ne prouve pas l'honnêteté.

Etapes

1. 1. Bookmark hebergeur / banque / admin.
2. 2. Compare un lien douteux (sans cliquer) a l'URL officielle.
3. 3. Sur ton site : active HTTPS si besoin, teste la redirection.
4. 4. Note ce que tu ferais si le navigateur affiche un gros warning.

Critere de reussite

Fiche de 3 sites remplie + regle personnelle "warning = stop".

◆ ASTUCE

Fais l'exercice avec quelqu'un : explique a voix haute ce que tu regardes.

Petite histoire

Sam a montre deux URLs presque identiques. Lea a trouve la lettre en trop. Max a admis le piege. Aucun clic. Atelier reussi.

Erreur classique

Ignorer un warning "juste pour voir la page". Non.

A toi

Ecris ta phrase stop. Colle-la mentalement avant chaque login critique.

Chapitre 17 - Atelier : checklist avant mise en ligne



Atelier : checklist avant mise en ligne.

Duree : 40 minutes. Reprends la checklist du mini-projet. Applique-la **avant** une mise en ligne ou une revue. Chaque case : fait / planifie (date) / non applicable. Chez DanielCraft, "planifie" sans date = non.

★ A RETENIR

HTTPS, comptes, maj, backup, droits, formulaires : cocher ou corriger.

Etapes

1. 1. Duplique la checklist 10 points.
2. 2. Remplis avec preuves courtes.
3. 3. Cree 3 tickets / taches pour les rouges.
4. 4. Planifie la revue J+90.

Critere de reussite

Document partage au responsable + au moins un rouge corrige le jour meme.

⚠ ATTENTION

Pas de scan d'intrusion improvise hors perimetre autorise.

Petite histoire

Lea a bloque une mise en ligne pour un plugin mort. Max a rage dix minutes puis a remercie. Sam a archive la checklist signee. Livraison propre.

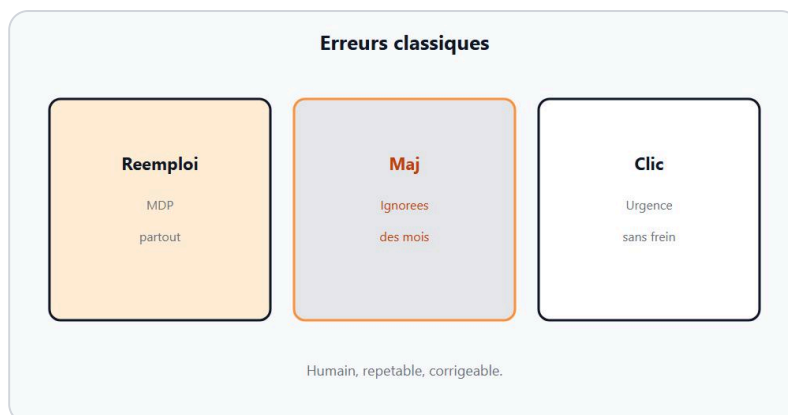
Erreur classique

Tout vert par fatigue. Fais honnete.

A toi

Envoie le document. Pose la date de revue dans le calendrier maintenant.

Chapitre 18 - Erreurs classiques



Erreurs classiques : reemploi MDP, maj ignorees.

Les accidents debutants se ressemblent. Chez DanielCraft, on les nomme pour les **eviter**, pas pour juger. Lea tient cette liste pres du bureau. Max s'y reconnait sans dramatique. Sam en fait un oral de cinq minutes en fin de module.

★ A RETENIR

Reconnaitre le piege. Appliquer le frein. Revenir a la carte.

Catalogue utile

1. 1. **MDP reutilises** - frein : gestionnaire + uniques.
2. 2. **Maj ignorees** - frein : calendrier + backup avant.
3. 3. **Backup jamais testee** - frein : restore mensuelle.
4. 4. **Admin pour tout le monde** - frein : roles.
5. 5. **Confiance aveugle au cadenas** - frein : verifier le domaine.
6. 6. **Clic sur urgence** - frein : URL tapee / favori.
7. 7. **Validation JS seule** - frein : serveur.
8. 8. **Comptes test en prod** - frein : supprimer.
9. 9. **Secrets dans le repo** - frein : coffre / variables d'env.
10. 10. **"Je suis trop petit"** - frein : bots ne lisent pas ton CA.

⚠ ATTENTION

Corriger une erreur ne demande pas d'apprendre a attaquer.

Petite histoire

Max a coche huit pieges sur dix "deja faits une fois". Lea a dit : "parfait, tu sais ou regarder". Sam a transforme la honte en plan. Chez DanielCraft, lucidite > perfection.

Erreur classique (meta)

Collectionner les listes sans rien changer. Choisis-en deux cette semaine.

♦ **ASTUCE**

Affiche les trois pieges qui te concernent. Barre-les quand c'est corrige.

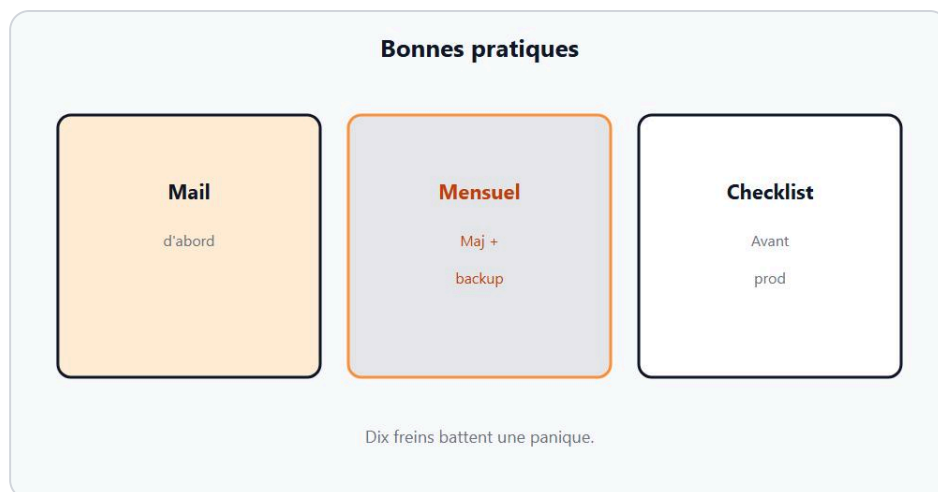
En vrai

Note ton top 3 personnel. Lie chaque item a un chapitre.

A toi

Traite le numero 1 de ton top 3 avant demain soir.

Chapitre 19 - Bonnes pratiques



Pratiques : freins simples, repetables.

Les bonnes pratiques securite debutant sont des **freins repetables**, pas une stack de buzzwords. Chez DanielCraft : ralentir, verifier, limiter, sauvegarder, mettre a jour, minimiser les donnees. Lea prefere dix minutes par semaine a une nuit de panique. Max a calé un creneau vendredi. Sam rappelle defense only.

★ A RETENIR

Ralentir, verifier, limiter, sauvegarder. Petit, clair, testable.

Routine hebdo (15 min)

- Regarder maj en attente
- Verifier backup recente
- Survoler comptes admin actifs
- Lire 2 messages douteux sans cliquer (entrainement frein)

Routine mensuelle

- Test restore
- Revue permissions
- Rotation si doute sur un secret
- Relire la checklist site

Ce qu'on ne fait pas ici

Pas d'exploits "pour s'entrainer". Pas de partage de dumps. Pas de scans sauvages. La curiosite se canalise vers la prevention et les sources defense.

◆ ASTUCE

Associe la routine a un rituel existant (vendredi cafe, fin de sprint).

Petite histoire

Sam a affiche la routine au mur de l'atelier. Lea l'a suivie trois mois : zero drame. Max a saute deux semaines et l'a paye par une maj urgente stressante. Le retour a la routine a suffi. DanielCraft mesure le succes au calme, pas au theatre.

Erreur classique

Acheter trois outils et ignorer la routine. L'outil sans habitude dort.

⚠ ATTENTION

La meilleure pratique reste inutile si elle n'a pas de date dans ton calendrier.

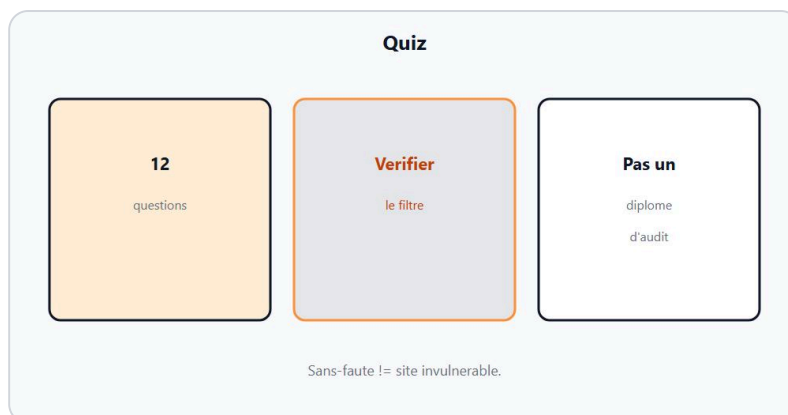
En vrai

Ecris ta routine A (hebdo) et B (mensuelle). Mets deux rappels.

A toi

Execute la routine A une fois aujourd'hui. Coche.

Chapitre 20 - Quiz



Quiz securite web.

Reponds **avant** de lire le corrigé. Chez DanielCraft, le quiz vérifie le frein, pas la recitation. Lea chronometre. Max écrit court. Sam interdit de tricher "juste pour voir".

★ A RETENIR

Reponds d'abord. Corrige ensuite. Note tes trous.

Questions

1. HTTPS protege surtout : (a) l'honnêteté du site (b) le trajet navigateur-serveur (c) tes plugins abandonnés ?
2. Un mot de passe unique sert à : (a) faire joli (b) limiter l'effet d'une fuite (c) éviter la 2FA ?
3. Face à un mail urgent "compte ferme, clique" : premier geste ?
4. Une sauvegarde non testée est : (a) suffisante (b) un espoir (c) inutile à jamais ?
5. Moindre privilège veut dire ?
6. Pourquoi valider côté serveur un formulaire ?
7. L'idée des requêtes préparées (sans détail d'attaque) ?
8. Le cadenas prouve-t-il que tu es sur le bon site ?
9. Cite deux signaux de phishing.
10. Que faire d'un compte stagiaire admin après le stage ?

Corrige court

1. (b) 2. (b) 3. Ne pas cliquer ; ouvrir via favori / URL tapée 4. (b) 5. Seulement les droits nécessaires 6. Le client n'est pas digne de confiance seul 7. Séparer structure de requête et valeurs / données 8. Non - vérifier le domaine 9. Urgence, faux domaine, demande de secret... 10. Révoquer / supprimer

⚠ ATTENTION

Si tu as rate 3+ questions, relis les chapitres concernés avant le bravo.

Petite histoire

Max a rate la question cadenas. Il a relu HTTPS dix minutes. Lea a valide. Sam a dit : "Mieux vaut ca qu'un clic."

A toi

Note ton score /10. Ecris trois actions liees aux erreurs.

Chapitre 21 - Bravo



Bravo. Tu proteges sans paniquer.

Tu as fini **Securite web - Les bases**. Tu sais reconnaitre sans paniquer, freiner un phishing, solidifier des mots de passe, lire HTTPS sans magie, planifier maj et sauvegardes, limiter les droits, comprendre l'idée des injections côté défense, contrôler un formulaire, soigner sessions et minimiser les données. Chez DanielCraft, c'est déjà un socle solide.

Lea continuerait par une revue à 90 jours. Max mettrait la routine dans le calendrier. Sam rappellerait : défense only, toujours. Toi, tu choisis le prochain frein à renforcer - pas le prochain tutoriel d'attaque.

★ A RETENIR

Habitudes > outils miracles. Tu proteges sans paniquer.

Ce que tu peux faire maintenant

- Appliquer la checklist à un vrai site
- Migrer le reste des comptes vers le gestionnaire
- Suivre la routine hebdo / mensuelle
- Relire un chapitre orange de ta carte

Ce que tu ne dois pas faire

Pas d'exploits "pour s'entraîner". Pas de tests sur des systèmes sans autorisation. La suite éventuelle (niveau intermédiaire) restera dans l'esprit prévention.

◆ ASTUCE

Envoie-toi un mail "revue securite - date" pour dans 90 jours avec ta checklist en pièce.

Petite histoire

En fin de module, Sam a éteint le projecteur. Lea a demandé un seul mot. Les réponses : freiner, unique, backup. Max a ajouté "calendrier". DanielCraft a clos : petit, clair, testable. Tu peux repartir.

A toi

Ecris une phrase de cloture personnelle. Garde-la. Puis execute une action concrete dans l'heure (2FA, backup, ou case rouge).

Tu as les briques. Maintenant, protege sans paniquer.